

Multicalibration for code LLMs: large gains from raw token probability, almost none from a decent model

Aarav Raina, October 2026. Code and lab notebook: github.com/aaravraina3/multicalibrating-llms

Summary

Campos et al. (2025) showed that multicalibration beats Platt scaling when you turn a code LLM's average token probability into a probability that its program passes its tests. I reimplemented their methods on their public dataset (CALIBRI, LiveCodeBench, Qwen3 Coder and GPT OSS) and asked three follow-up questions.

- **Replication.** My code reproduces their Table 1 to the printed three decimals for every method except IGLB (within 0.004). The gap between their paper and their released results file comes from how group thresholds are computed. Their stated reason for IGHB's weak numbers (overfitting) doesn't hold in their code: it stops too early.
- **Starting score.** When the starting score is a logistic model on cheap label free features, multicalibration adds nothing over Platt. On test, IGLB minus Platt in Brier skill score (BSS) is +0.593 [0.522, 0.668] for GPT OSS starting from token probability, and -0.002 [-0.008, 0.003] starting from the feature model.
- **Routing.** Multicalibration only changes routing decisions when the starting score is weak. When running both models and keeping the more confident answer, IGLB beats Platt by 6.7 points of pass rate from token probability [4.5, 8.9] and by 0.1 [-0.2, 0.5] from the feature model.
- **Overfitting.** With little calibration data, IGHB fixes noise. A two standard error rule, computed by problem, removes most of the damage.

1. Question

A code assistant that knows how likely its answer is to pass can decide when to show it, when to retry, and when to call a stronger model. Token probabilities give a confidence for free, but they're badly overconfident: Qwen3 Coder's average token probability is 0.88 on validation, while its pass rate is 0.45.

Calibration fixes the overall scale. Multicalibration (Hébert-Johnson et al., 2018) asks for more: the probabilities should also be right inside every identifiable group, such as long outputs or hard problems. Campos et al. found multicalibration methods beat Platt scaling on code generation. Their starting score was always raw token probability, which knows nothing about the groups. So I asked:

1. **RQ1.** Do I get their numbers with my own implementation?

2. **RQ2.** Do multicalibration's gains over Platt survive when the starting score is a feature rich model instead of raw token probability?
3. **RQ3.** Does multicalibration improve decisions about routing between two models, beyond Platt?
4. **RQ4** (exploratory). With small calibration sets, does iterative patching overfit, and can a noise aware stopping rule or a reusable holdout stop it?

2. Data

CALIBRI (Campos et al., 2025), LiveCodeBench release v6: 1055 competitive programming problems, split by problem into 527 train, 264 validation, 264 test. Each problem has 10 generations from Qwen3 Coder 30B A3B and from GPT OSS 20B, with every token's log probability and a pass/fail label from the benchmark's tests.

One property of the data drives many results: **generation was capped at 2000 tokens**, and both models are reasoning models. 27% of Qwen3 outputs and 44% of GPT OSS outputs hit the cap, almost always before writing code. Truncated outputs pass 0.4% (Qwen3) and 0.6% (GPT OSS) of the time. GPT OSS outputs that finish pass 92% of the time. Average token probability barely reflects truncation, so a lot of what any calibrator can gain on GPT OSS is "notice that the model never finished".

3. Methods

Starting scores. `avg_prob` is the geometric mean token probability over the whole output, reasoning included (Campos's best score). The feature models are fit on 316 train problems (`base_train`): B1 is a logistic regression on size, token log probability statistics (all tokens and code tokens), and Python AST structure; B2 adds self consistency (similarity of a sample's code to the other 9 samples of the same problem, using only code text); B3 is gradient boosting on the B2 features. Hyperparameters are picked on validation. No base model sees the difficulty label.

Calibrators. Platt scaling; histogram binning (HB); LINR (one linear regression that fixes every group's average); IGHb (repeatedly shift the group x confidence bin cell with the largest share x squared error); IGLB (same loop over larger overlapping cells, patched with a small logistic curve and stopped when held out Brier stops improving). For RQ2 to RQ4 they're fit on the other 211 train problems (`calib`). IGLB early stops on 63 of them, so validation and test stay untouched by every method.

Groups. For replication, the six Campos groups: long code, long prompt, long output, and easy/medium/hard. For RQ2 and RQ3, a deployment realistic set without difficulty: long code, long prompt, long output, deeply nested code, uses imports, syntax invalid (Qwen3 only), truncated, and all rows. A second set adds difficulty. Thresholds come from train only.

Protocol. Every setting was chosen on validation and frozen in `PROTOCOL.md` before a single test run (tag `v-final`, commit `8f1de29`). Each RQ has one named primary comparison. Intervals are 95% task clustered paired bootstrap intervals: resample problems, never rows, 2000 times. The lab notebook logs every decision and bug.

4. Results

RQ1: replication

Method	Qwen3 BSS (paper)	Qwen3 BSS (mine, 95% interval)	GPT OSS BSS (paper)	GPT OSS BSS (mine, 95% interval)
uncalibrated	-0.543	-0.543 [-0.738, -0.378]	-0.075	-0.075 [-0.165, -0.008]
Platt	0.377	0.377 [0.315, 0.431]	0.187	0.187 [0.136, 0.232]
HB	0.383	0.383 [0.315, 0.443]	0.237	0.237 [0.178, 0.292]
LINR	0.463	0.463 [0.389, 0.532]	0.721	0.721 [0.667, 0.766]
LOGR	0.306	0.306 [0.163, 0.439]	0.733	0.733 [0.657, 0.800]
IGHB	0.232	0.232 [0.117, 0.330]	0.412	0.412 [0.348, 0.469]
IGLB	0.480	0.478 [0.394, 0.557]	0.764	0.768 [0.707, 0.820]

Three things came out of matching their code line by line:

- **Median convention.** Their released code computes the length group medians over train, validation, and test pooled. With pooled medians I reproduce their released results file to six decimals on 13 of 14 numbers. With train only medians I reproduce the paper's table above. So the paper's table appears to come from an earlier version that used train only medians.
- **LOGR outputs hard labels.** Their LOGR calls `predict` instead of `predict_proba`, so it outputs 0 or 1. Its Brier score is then exactly one minus accuracy, which explains its odd row: high accuracy, middling BSS.
- **IGHB is underfit, not overfit.** The paper attributes IGHb's weak numbers to overfitting. In their code IGHb stops when every group's weighted error is below $1/M = 0.05$, which is loose. It stops after 3 to 5 updates with train BSS as low as validation BSS (Figure 1). Tightening the threshold to 0.001 raises validation BSS from 0.18 to 0.46 (Qwen3) and from 0.48 to 0.80 (GPT OSS).

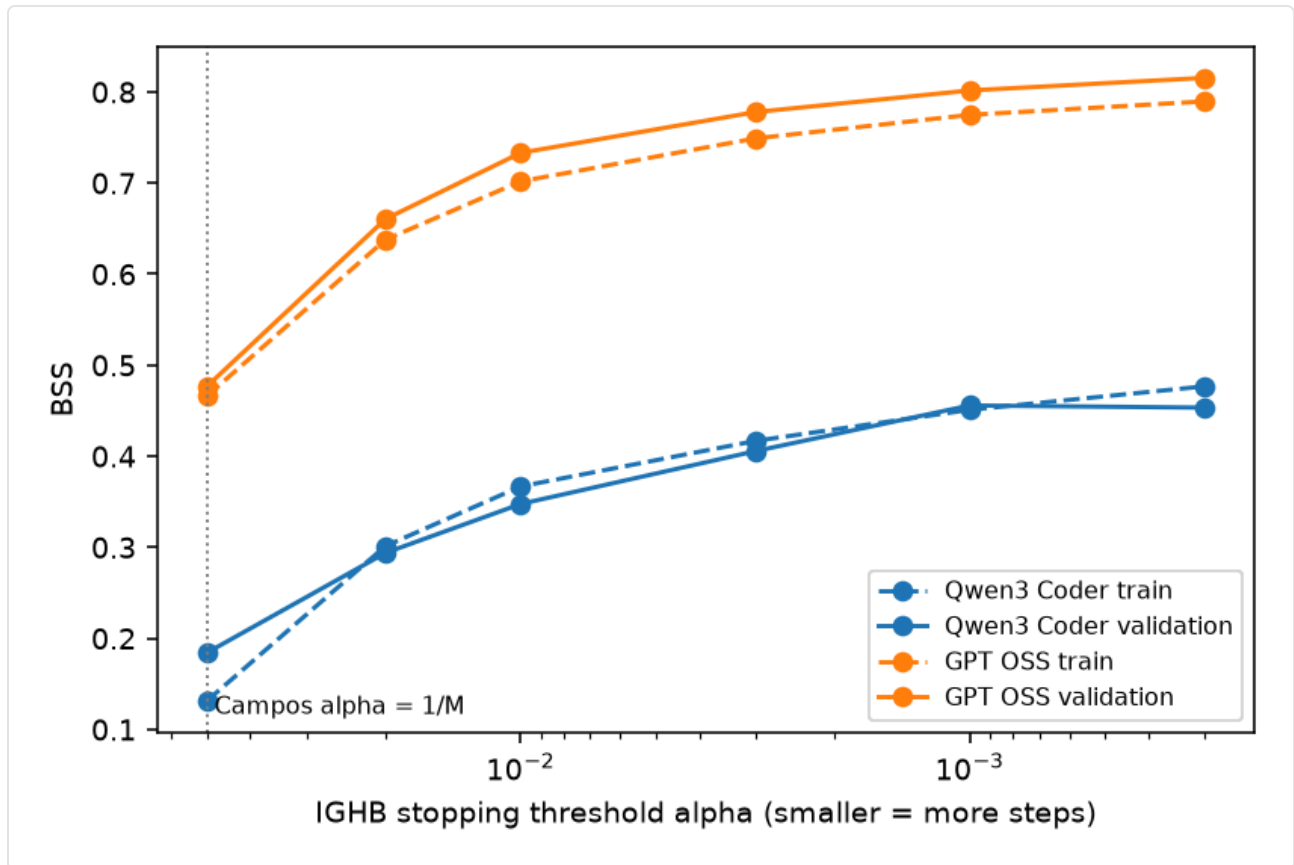


Figure 1. IGHB train and validation BSS against its stopping threshold. At the Campos threshold (dotted line) train and validation are equally bad: the method stops early. Overfitting shows up only at much smaller thresholds.

The bootstrap intervals are about ± 0.07 wide, so differences between neighboring methods in the table, like LINR vs IGLB or Platt vs HB, are within noise on 264 test problems.

RQ2: does a better starting score leave anything to fix?

Test BSS with the deployment realistic groups:

Model	Start	Uncalibrated	Platt	HB	LINR	IGHB	IGLB
Qwen3	avg_prob	-0.543	0.390	0.382	0.452	0.401	0.409
Qwen3	B1	0.570	0.567	0.553	0.565	0.552	0.570
Qwen3	B2	0.560	0.559	0.540	0.560	0.527	0.560
Qwen3	B3	0.520	0.519	0.514	0.539	0.512	0.520
GPT OSS	avg_prob	-0.075	0.197	0.213	0.773	0.762	0.790
GPT OSS	B1	0.818	0.821	0.817	0.817	0.816	0.816
GPT OSS	B2	0.820	0.823	0.816	0.819	0.816	0.821
GPT OSS	B3	0.816	0.819	0.820	0.817	0.814	0.816

Primary comparison, IGLB minus Platt on test:

Model	From avg_prob	From B2
Qwen3	+0.019 [-0.016, 0.054]	+0.001 [-0.008, 0.012]
GPT OSS	+0.593 [0.522, 0.668]	-0.002 [-0.008, 0.003]

The hypothesis holds where there was something to gain. From raw token probability, multicalibration adds 0.6 BSS over Platt on GPT OSS, because the groups carry the truncation signal the score lacks. From any feature based start, no multicalibration method beats Platt by more than 0.02. The uncalibrated feature models are already about as good as Platt on top of them, and IGHB is clearly worse on Qwen3 B2 (-0.033 [-0.063, -0.005]). A logistic model trained with log loss is already close to calibrated along its own features, and every group here is built from those features (Figure 2).

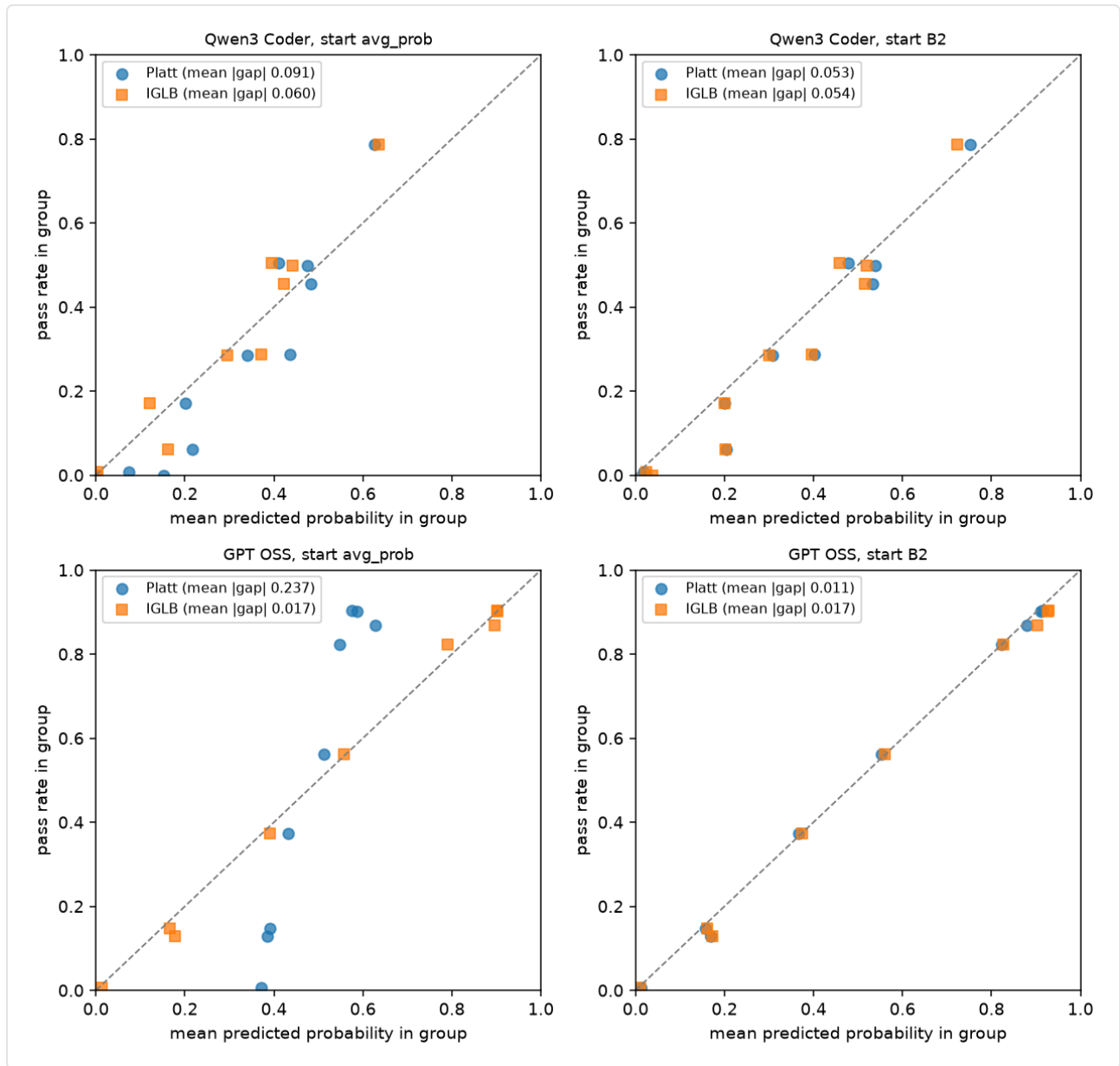


Figure 2. Mean prediction vs pass rate for each group on test. From avg_prob, Platt leaves GPT OSS groups far off the diagonal and IGLB fixes them. From B2, the two coincide.

Two secondary results qualify this.

- Difficulty is new information.** Adding difficulty groups helps Qwen3 even on top of B2 (LINR +0.041 [0.010, 0.077]), because the base model never saw difficulty. In Figure 2, Qwen3's two off-diagonal groups under B2 are hard problems (predicted 0.20, pass rate 0.06) and code that uses imports (0.40 vs 0.29). A model trained with difficulty as a feature would likely absorb the first. The imports gap is in IGLB's group set, but IGLB made no patches on B2: on its 63 problem early stopping set, no patch lowered Brier.

- **Qwen3 gains little even from avg_prob** with the realistic groups (IGLB +0.019 [-0.016, 0.054], LINR +0.062 [0.025, 0.102]). Its failures are less about truncation, so the groups say less.

Excluding empty programs, the Qwen3 conclusions are the same (IGLB minus Platt +0.018 from avg_prob, +0.002 from B2). For GPT OSS, rows with code pass 94% of the time and no starting score reaches BSS 0.1 on them: its whole calibration problem is recognizing outputs without code.

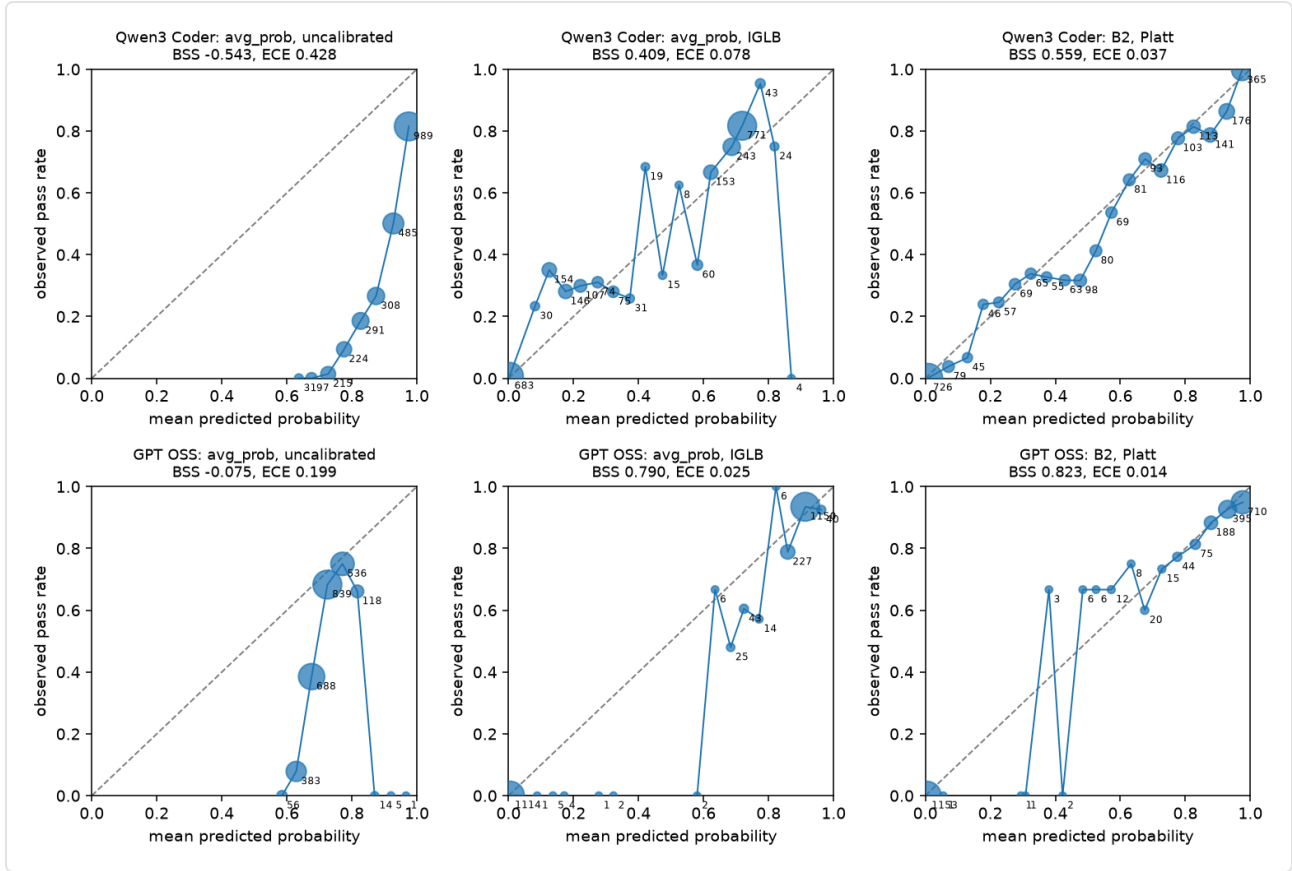


Figure 3. Test reliability diagrams. Raw token probability is overconfident; IGLB on avg_prob fixes the overall shape; Platt on B2 is close to the diagonal and spreads predictions over the whole range.

RQ3: does multicalibration change routing decisions?

Each test row pairs sample k of Qwen3 (primary, fewer output tokens) with sample k of GPT OSS (fallback). Always using Qwen3 passes 45.8%, always GPT OSS 51.1%, and an oracle that escalates exactly when it helps 61.4%.

Policy (test pass rate)	Start	Uncalibrated	Platt	IGLB
Escalate lowest 20% of primary confidence	avg_prob	0.484	0.484	0.486
Escalate lowest 20% of primary confidence	B2	0.481	0.481	0.481
Run both, keep the more confident answer	avg_prob	0.469	0.524	0.591
Run both, keep the more confident answer	B2	0.596	0.595	0.597

- **Rank based escalation.** Platt can't change which rows are escalated, because it preserves order. That was checked: zero different decisions in all 16 cases. Multicalibration can reorder rows across groups. That helps when GPT OSS is primary and starts from avg_prob (+2.2 points [0.5, 4.1] at 20% escalation), and does nothing from B2.
- **Comparing two models' probabilities** is where calibration matters most, because the two numbers have to share a scale. From avg_prob, Platt lifts the pass rate from 46.9% to 52.4% and IGLB to 59.1%, within 2.3 points of the oracle (primary comparison: +6.7 [4.5, 8.9]). From B2 all three are within 0.2 points (+0.1 [-0.2, 0.5]).
- **Fixed cost thresholds were mixed.** Escalating when the primary's probability falls below $q_B - c_B / L$ (assumed costs: c_B in {1, 2, 5}, $L = 10$) sometimes favored IGLB and sometimes Platt. The threshold also needs the fallback's chance on this particular problem. The two models fail on the same hard problems, so the fallback's overall pass rate overstates its chances exactly where the primary looks worst. Better calibration of the primary alone doesn't fix that.

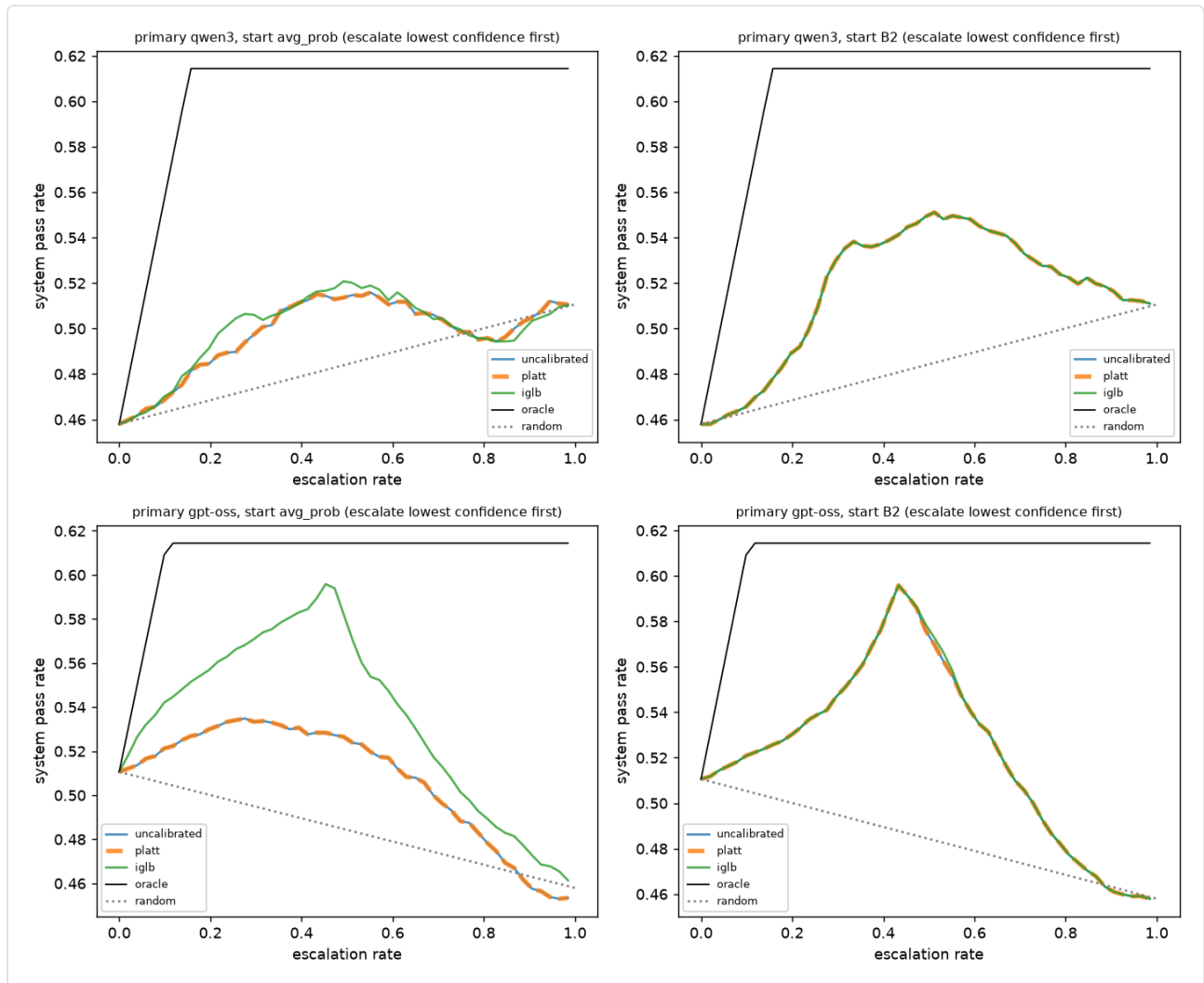


Figure 4. System pass rate against escalation rate when escalating the least confident primary answers first (test). Platt's curve sits exactly on the uncalibrated one. IGLB lifts the curve only from avg_prob.

RQ4: overfitting with small calibration sets

I subsampled the calibration set to 25 to 211 problems (20 repeats) and measured how much each method changed test Brier relative to its starting score. Starting from B2, there's little to fix, so any change is mostly fitted noise.

Method (test Brier change from B2, n = 25 problems)	Qwen3	GPT OSS
Platt	+0.008	+0.001
IGLB	+0.004	+0.002
IGHB, chosen threshold	+0.063	+0.015
IGHB, run long	+0.086	+0.028
IGHB, holdout checked with noise	+0.061	+0.017
IGHB, noise aware (gap > 2 SE by problem)	+0.007	+0.001

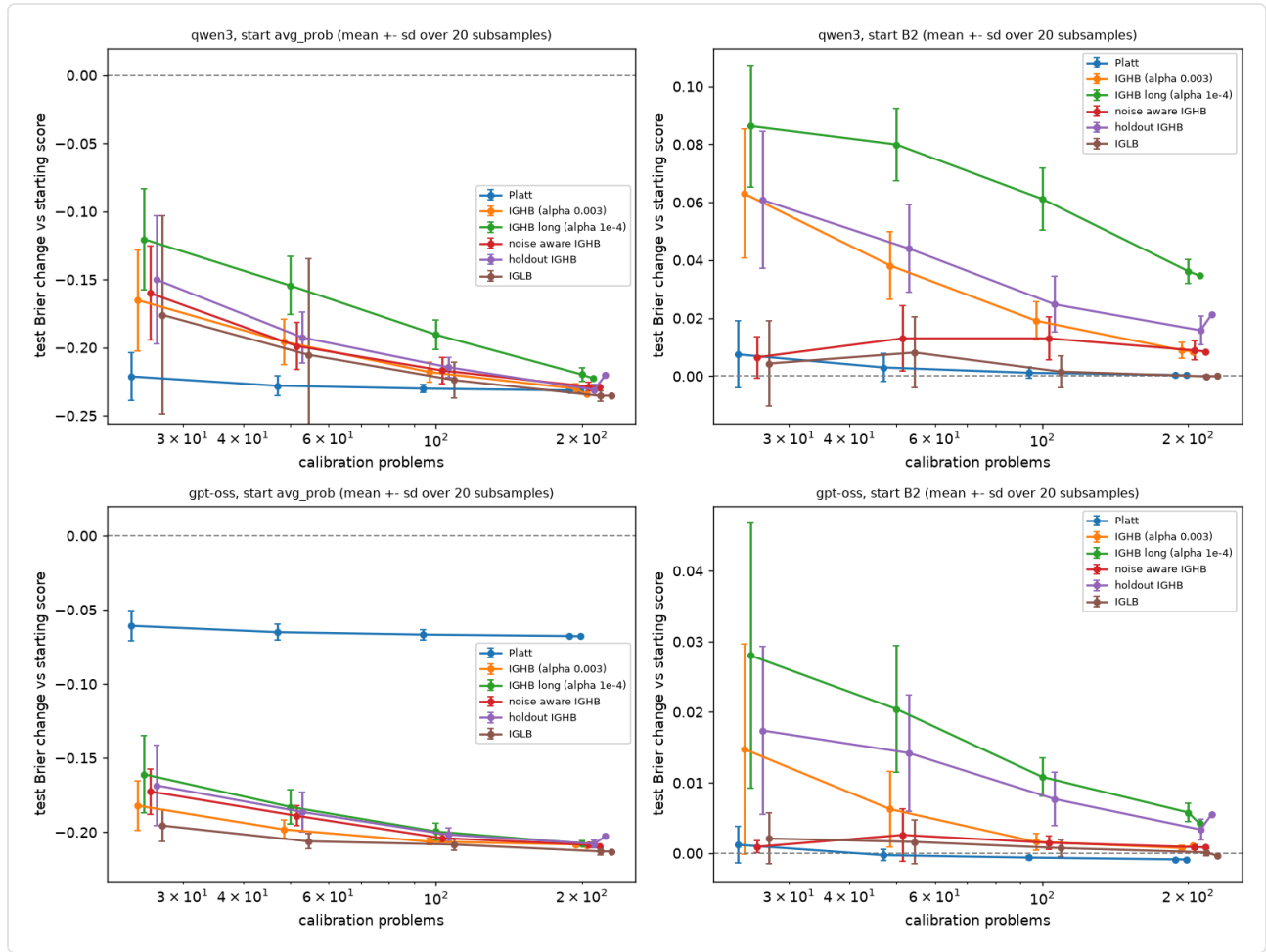


Figure 5. Brier change against calibration set size (mean and standard deviation over 20 subsamples). Below zero means the method helped.

IGHB overfits small calibration sets, and the harm shrinks as data grows. The noise aware rule, which only updates a cell whose gap exceeds two standard errors with problems as the unit, makes IGHB about as safe as Platt and IGLB. It gives up little when there is real signal (from avg_prob, Qwen3 at $n = 25$: -0.160 vs -0.165). The holdout check helps less: it spends 30% of an already small sample, and one noisy Brier comparison accepts many bad updates. The noise aware rule needed a minimum of 5 problems per cell, added after seeing validation results; this RQ is exploratory.

5. Limitations

- One benchmark, two models of similar size, public data. I didn't generate or execute code myself, and the labels come from benchmark tests, which aren't proof of correctness.
- The 2000 token cap shapes the data. Many GPT OSS results are about recognizing truncated outputs, which a deployment with a larger budget might not have.

- The calibration set is small (211 problems, 63 of them for IGLB's early stopping). Group methods might do better with more data.
- Self consistency assumes 10 samples per problem at inference, ten times the generation cost.
- Difficulty labels are benchmark metadata a real assistant wouldn't have. Main results exclude them.
- Routing costs are assumptions. Changing them can flip which calibrator wins on cost based policies.
- Empty programs dominate GPT OSS. Results are reported with and without them.
- The bootstrap treats fitted calibrators as fixed. It doesn't capture variation from refitting on different calibration data.
- The groups are hand made. A shallow tree on B2's residuals found a structure interaction for Qwen3 (`runs/phase9/tree_auditor.txt`) that no hand made group covers.

6. Related work

- Hébert-Johnson, Kim, Reingold, Rothblum (2018) define multicalibration and the iterative fixing algorithm, and show that multicalibrating a good model can't hurt it much. Their Section 3.3 handles adaptive reuse of the sample with a noisy oracle.
- Detommaso, Bertran, Fogliato, Roth (2024) bring multicalibration to LLM confidence scores and introduce IGHB and IGLB.
- Campos, Kuschner, Ulges (2025) apply them to code generation and release CALIBRI. This study replicates and extends their LiveCodeBench results.
- Hansen, Devic, Nakkiran, Sharan (2024) find that models calibrated out of the box tend to already be multicalibrated, and that post processing mostly helps uncalibrated models. RQ2 is the code LLM version of that finding.
- Globus-Harris, Harrison, Kearns, Roth, Sorrell (2023) show multicalibration is a form of boosting, which explains both its power and its overfitting.
- Dwork et al. (2015) (Thresholdout) and Feldman and Steinke (2018) control overfitting from adaptive reuse of data with noise. The holdout and noise aware IGHB variants are simple versions of these ideas.
- Spiess et al. (2025) compare Platt scaling with self reflection for code LLM calibration.

Reproduce: see the README.